A PROMPT PLAYBOOK

IDEA TO PROTOTYPE

The 20-prompt playbook for building software with an AI coding agent.

MADE BY ISH
Cooked with Diet Coke

CONTENTS

Introduction	03
How to use this book	04
The workflow at a glance	05
01 Lay the foundations	07
02 Plan and set up	14
03 Build the product	17
04 Test, debug and harden	23
05 Ship and watch	28
Quick reference	32
Conclusion	33

INTRODUCTION

Most AI-built projects start with an idea and a request for code. That works for a demo and falls apart by the second week, because the agent has nothing fixed to refer back to. Every new session makes fresh guesses about the design, the database and the way errors are handled, and the guesses do not agree with each other.

This book takes the opposite route. Before any code is written, the idea is turned into a small set of documents that the agent treats as the source of truth. The same agent then plans, builds, tests, secures, deploys and audits the product against those documents. The whole path is twenty prompts, run in order.

The prompts are written to be pasted as they are. Each gives the agent a role, the context to read, a task, a short list of rules and the output it must report back. The rules matter most: they keep the agent from inventing features, adding dependencies nobody asked for, or rewriting code that already works.

FIRST HALF	SECOND HALF
DOCUMENTS Decide what you are building, once, and write it down.	**EXECUTION** Build, check and ship against those documents.

The book follows five chapters in the order the work happens, from the first document to the final launch verdict. Prompt by prompt, you will see what each one reads, what you fill in, and what you get back.

HOW TO USE THIS BOOK

1 **Work in order.**
Prompts 1 to 6 create documents, and later prompts read them. Skipping ahead leaves the agent guessing again.

2 **Replace the square brackets.**
Five prompts have a placeholder for you to fill: [PROJECT IDEA], [TASK / FEATURE], [MCP SERVER / PURPOSE], [SERVICE / API NAME + PURPOSE] and [ERROR / BUG / LOG / SCREENSHOT].

3 **Save each document in your project.**
When a prompt asks for Markdown ready to save, save it under the file name it gives, such as PRD.md, so the next prompt can read it.

4 **Read before you approve.**
The agent is asked to label its assumptions and flag decisions that need you. Those flags are yours to resolve; the documents are only as good as your review.

5 **Repeat what repeats.**
Prompts 9, 12, 13 and 15 are the ones you will run again and again. Prompts 14 to 17 are worth running again before each release.

THE ANATOMY OF EVERY PROMPT

Role	Tells the agent which kind of expert to be, so its choices fit the job: architect, QA engineer, SRE.
Context	Names the documents to read first, or the task to carry out. This keeps the agent tied to your source of truth.
Task	What to produce or do, usually with a checklist of what to cover.
Rules	The limits: no invented features, no hardcoded secrets, no unrelated changes, and stop and ask when something blocks.
Output	The exact shape of the answer: a clean Markdown file, or a report of what changed and what was tested.

THE WORKFLOW AT A GLANCE

Twenty prompts in five chapters. The first chapter writes the documents; everything after it works from them.

---

01 LAY THE FOUNDATIONS

01 PRD Creation · 02 DESIGN.md · 03 ARCHITECTURE.md · 04 DATABASE.md · 05 API.md · 06 AGENTS.md / Coding Rules

---

02 PLAN AND SET UP

07 Implementation Planning · 08 Project Setup

---

03 BUILD THE PRODUCT

09 Feature Development · 10 Database & Backend Connection · 11 Authentication & Authorization · 12 MCP Connection · 13 External API / Service Integration

---

04 TEST, DEBUG AND HARDEN

14 Testing & QA · 15 Debugging / Root Cause · 16 Security Audit & Hardening · 17 Code Review & Refactoring

---

05 SHIP AND WATCH

18 Deployment & Production Setup · 19 Production Monitoring & Maintenance · 20 Final Production Audit

---

RUN ONCE, RUN OFTEN

Prompts 1 to 8 are run once for a project. Prompts 9, 12, 13 and 15 repeat as the product grows, once per feature, tool, service or bug. Prompts 14 to 17 suit running again before each release, and prompts 18 to 20 take the product live.

WHAT EACH PROMPT READS

A dot means the prompt tells the agent to read that document. Each column is the file of the same name, so ARCH is ARCHITECTURE.md.

#	PROMPT	PRD	DESIGN	ARCH	DATABASE	API	AGENTS
01	PRD Creation						
02	DESIGN.md	•					
03	ARCHITECTURE.md	•	•				
04	DATABASE.md	•		•			
05	API.md	•		•	•		
06	AGENTS.md / Coding Rules	•	•	•	•	•	
07	Implementation Planning	•	•	•	•	•	•
08	Project Setup	•	•	•			•
09	Feature Development	•	•	•	•	•	•
10	Database & Backend Connection			•	•	•	•
11	Authentication & Authorization	•		•	•	•	•
12	MCP Connection						•
13	External API / Service Integration			•	•	•	•
14	Testing & QA	•		•		•	
15	Debugging / Root Cause						•
16	Security Audit & Hardening	•		•	•	•	•
17	Code Review & Refactoring	•	•	•	•	•	•
18	Deployment & Production Setup			•	•	•	•
19	Production Monitoring & Maintenance			•			
20	Final Production Audit	•	•	•	•	•	•

Prompt 01 reads nothing, because it starts from your idea. Prompts 14 to 17 and 20 also work against the current code, and prompt 19 reviews the deployed application.

CHAPTER 1 · PROMPTS 01–06

LAY THE FOUNDATIONS

Six prompts, six documents. Each one asks the agent to read the documents before it, so the set stays consistent: the architecture respects the design, the database respects the architecture, and the rules in AGENTS.md respect all of them.

Read each document before you approve it. The prompts ask the agent to label assumptions and flag decisions that need you, and those flags are where your attention belongs. Every later prompt points back to this chapter.

IN THIS CHAPTER

01 PRD Creation	08
02 DESIGN.md	09
03 ARCHITECTURE.md	10
04 DATABASE.md	11
05 API.md	12
06 AGENTS.md / Coding Rules	13

PROMPT 01 · CHAPTER 1

PRD CREATION

Run this first, with a clear description of what you want to build. The PRD fixes the scope, so later prompts do not have to guess it.

READS	YOU FILL IN	YOU GET
Nothing yet	[PROJECT IDEA]	PRD.md

THE PROMPT

You are a senior Product Manager and Software Architect experienced in taking products from idea to production.

CONTEXT:

I want to build: [PROJECT IDEA]

TASK:

Create a complete PRD.md that will become the source of truth for the coding agent.

Include:

- Problem, value proposition and target users
- Core user journeys and user stories
- Features and functional requirements
- Non-functional requirements
- Pages/screens
- Roles and permissions
- Data requirements
- Integrations
- Loading, empty and error states
- MVP scope, out-of-scope items and success criteria

RULES:

Do not write code. Do not invent unnecessary features. Avoid over-engineering. Clearly label assumptions and flag decisions requiring my approval.

OUTPUT:

Return only clean Markdown ready to save as PRD.md.

PROMPT 02 · CHAPTER 1

DESIGN.MD

Run once the PRD is approved. It turns the product requirements into visual and interaction rules the agent can reuse, instead of restyling every screen from scratch.

READS	YOU FILL IN	YOU GET
PRD.md	Nothing	DESIGN.md

THE PROMPT

You are a senior Product Designer and UX/UI Design Systems expert specializing in production web applications.

CONTEXT:

Use PRD.md as the product source of truth.

TASK:

Create DESIGN.md as the single source of truth for the application's visual design and UX.

Define:

- Visual direction and design principles
- Colors and typography
- Spacing and layout system
- Reusable components
- Navigation and forms
- Loading, empty and error states
- Interactions and animations
- Responsive behavior
- Accessibility requirements
- Page-level layout guidance

RULES:

Prioritize usability and consistency. Avoid generic AI-generated SaaS design and unnecessary decoration. Define reusable rules, not one-off styling.

OUTPUT:

Return only clean Markdown ready to save as DESIGN.md. Do not write code.

ARCHITECTURE.MD

This fixes the technology and the structure before any code exists. Read the stack choices carefully, because every later prompt is told to follow them.

READS	YOU FILL IN	YOU GET
PRD.md, DESIGN.md	Nothing	ARCHITECTURE.md

THE PROMPT

You are a senior Software Architect specializing in secure, scalable and maintainable applications.

CONTEXT:

Use PRD.md and DESIGN.md as the source of truth.

TASK:

Create ARCHITECTURE.md defining exactly how this product should be built.

Include:

- Recommended tech stack with reasoning
- Frontend/backend architecture
- Data flow
- Folder structure
- Database interaction
- Authentication/authorization
- API architecture
- External services
- AI architecture if needed
- Environment configuration
- Error handling
- Security boundaries
- Testing and deployment approach

RULES:

Choose the simplest reliable architecture. Avoid unnecessary dependencies, microservices or infrastructure. Separate MVP requirements from future scalability.

OUTPUT:

Return only clean Markdown ready to save as ARCHITECTURE.md. Do not write code.

DATABASE.MD

This turns the user journeys into a data model: tables, relationships, access rules and the handling of sensitive data.

READS	YOU FILL IN	YOU GET
PRD.md, ARCHITECTURE.md	Nothing	DATABASE.md

THE PROMPT

You are a senior Database Architect and Backend Engineer experienced in designing production-grade application databases.

CONTEXT:

Use PRD.md and ARCHITECTURE.md as the source of truth.

TASK:

Create DATABASE.md defining the complete data model.

Include:

- Database technology
- Tables and columns with types
- Primary/foreign keys
- Relationships and ER diagram
- Constraints and validation
- Indexes based on real queries
- Ownership and authorization
- Row-level security where applicable
- Sensitive-data handling
- Migration and seed strategy
- Backup/recovery considerations

RULES:

Design only required data. Avoid duplication, unnecessary tables and premature optimization. Ensure the schema supports the defined user journeys and security model.

OUTPUT:

Return only clean Markdown ready to save as DATABASE.md. Do not write application code.

API.MD

This defines every internal endpoint and external integration before it is built, so the frontend, the backend and the tests all agree.

READS	YOU FILL IN	YOU GET
PRD.md, ARCHITECTURE.md, DATABASE.md	Nothing	API.md

THE PROMPT

You are a senior Backend Architect and API Integration Engineer experienced in production systems.

CONTEXT:

Use PRD.md, ARCHITECTURE.md and DATABASE.md as the source of truth.

TASK:

Create API.md defining how the application communicates internally and with external services.

Include:

- Endpoints, methods and purpose
- Authentication/authorization
- Request/response formats
- Validation and error conventions
- Pagination/filtering where needed
- External APIs and integrations
- Webhooks and verification
- Rate limiting
- Required environment variables
- Data privacy and failure handling

RULES:

Define only required APIs. Never expose secrets client-side. Enforce authorization server-side. Keep conventions consistent and integrations fault-tolerant.

OUTPUT:

Return only clean Markdown ready to save as API.md. Do not write code.

AGENTS.MD / CODING RULES

The last document, and the one the agent consults most. It sets the working rules for every later session. Check which file name your coding tool reads for project rules, and save the content under that name as well.

READS	YOU FILL IN	YOU GET
PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md	Nothing	AGENTS.md

THE PROMPT

You are a senior software engineer and AI coding-agent specialist experienced in maintaining large production codebases.

CONTEXT:

Use PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md and API.md as the project source of truth.

TASK:

Create AGENTS.md containing the rules the coding agent must follow throughout development.

Define rules for:

- Code quality and naming
- Project/folder structure
- Component reuse
- Dependency management
- Type safety
- Error handling
- Security and secrets
- Database/API access
- Testing
- Documentation
- Git/change discipline

RULES:

The agent must inspect existing code before changing it, follow the project documents, reuse existing patterns, avoid unnecessary rewrites/dependencies, never expose secrets, and avoid unrelated changes.

OUTPUT:

Return only clean Markdown ready to save as AGENTS.md.

CHAPTER 2 · PROMPTS 07–08

PLAN AND SET UP

With the documents approved, turn them into a plan and an empty but working project. One prompt produces an ordered list of small phases without writing code. The other builds only the skeleton, with no product features.

Keeping the two apart stops the agent from starting on features before the foundations exist.

IN THIS CHAPTER

07 Implementation Planning 15

08 Project Setup 16

IMPLEMENTATION PLANNING

This turns the six documents into an ordered build plan made of small tasks you can verify one at a time. The plan ends with a recommended first task.

READS	YOU FILL IN	YOU GET
All six documents	Nothing	An ordered implementation plan

THE PROMPT

You are a senior Technical Lead responsible for planning production software development.

CONTEXT:

Read PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md before planning.

TASK:

Create a detailed implementation plan for this project WITHOUT writing application code.

Break development into small, logical phases. For each phase define:

- Goal
- Features/tasks
- Files or modules likely affected
- Dependencies/prerequisites
- Database/API changes
- Tests required
- Completion criteria

RULES:

Build foundations before dependent features. Keep tasks small enough to implement and verify independently. Follow the architecture and MVP scope. Do not introduce features or technologies not specified in the project documents.

OUTPUT:

Return an ordered implementation plan with clear phases and tasks. End with the recommended first task.

PROJECT SETUP

This creates the empty but runnable project: framework, folders, configuration, scripts and checks. No product features are built yet.

READS	YOU FILL IN	YOU GET
PRD.md, DESIGN.md, ARCHITECTURE.md, AGENTS.md	Nothing	A runnable project skeleton

THE PROMPT

You are a senior full-stack engineer responsible for establishing production-ready project foundations.

CONTEXT:

Read PRD.md, DESIGN.md, ARCHITECTURE.md and AGENTS.md before making changes.

TASK:

Initialize the project according to the approved architecture.

Set up:

- Required framework and dependencies
- Folder structure
- Configuration
- Environment variable structure
- Base routing
- Core styling/design system
- Shared utilities/components
- Development scripts
- Linting/formatting/type checking
- Git-ready project structure

RULES:

Follow the documented architecture exactly. Inspect the existing project first if files already exist. Do not add unnecessary packages or build application features yet. Never hardcode secrets. Keep the project runnable after setup.

OUTPUT:

Implement the setup, verify it runs successfully, and briefly report what was created, commands used and any decisions requiring approval.

CHAPTER 3 · PROMPTS 09–13

BUILD THE PRODUCT

Five prompts that do the building. Prompt 9 is the one you will repeat: one run for each task in your plan. Prompts 10 and 11 set up the backend, the database and sign-in.

Prompts 12 and 13 reach outside the project: to tools through MCP, and to external services through their APIs. Each prompt tells the agent to inspect the existing code first and to keep its changes focused.

IN THIS CHAPTER

09 Feature Development	18
10 Database & Backend Connection	19
11 Authentication & Authorization	20
12 MCP Connection	21
13 External API / Service Integration	22

FEATURE DEVELOPMENT

The prompt you will use most. Give it one task from your plan and it builds that feature with tests, following the documents.

READS	YOU FILL IN	YOU GET
All six documents	[TASK / FEATURE]	A tested feature and a change report

THE PROMPT

You are a senior full-stack engineer working inside an existing production codebase.

CONTEXT:

Follow PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md. Implement only this approved task:

[TASK / FEATURE]

TASK:

First inspect the relevant existing code and dependencies. Briefly determine the implementation approach, then implement the feature completely.

Requirements:

- Follow existing architecture and design system
- Reuse existing components/utilities
- Handle loading, empty, success and error states
- Validate inputs and enforce authorization
- Update database/API code when required
- Add or update relevant tests
- Keep changes focused

RULES:

Do not rewrite unrelated code or introduce unnecessary dependencies. Do not silently change requirements. If a blocking ambiguity exists, stop and ask.

OUTPUT:

Implement, test, and report changed files, tests performed, and any remaining issues.

PROMPT 10 · CHAPTER 3

DATABASE & BACKEND CONNECTION

This connects the real database and backend: schema, migrations, data access, endpoints, validation and tests.

READS	YOU FILL IN	YOU GET
DATABASE.md, ARCHITECTURE.md, API.md, AGENTS.md	Nothing	A working backend and database

THE PROMPT

You are a senior backend engineer and database specialist experienced in production systems.

CONTEXT:

Follow DATABASE.md, ARCHITECTURE.md, API.md and AGENTS.md as the source of truth.

TASK:

Connect and implement the application's backend and database.

Do the following:

- Configure the database securely
- Create the approved schema and migrations
- Implement models/data access
- Implement required backend services/API endpoints
- Add validation and authorization
- Handle transactions and errors
- Add appropriate indexes
- Add seed/development data if specified
- Add tests for critical database and backend behavior

RULES:

Never hardcode credentials. Follow the approved schema. Do not modify database structure without justification. Enforce authorization server-side. Keep secrets in environment variables.

OUTPUT:

Implement and verify the connection and core backend functionality. Report migrations, endpoints/services created, tests run, and any issues requiring approval.

AUTHENTICATION & AUTHORIZATION

This adds sign-in, sessions, roles and ownership checks, all enforced on the server rather than in the browser.

READS	YOU FILL IN	YOU GET
PRD.md, ARCHITECTURE.md, DATABASE.md, API.md, AGENTS.md	Nothing	Authentication, authorization and tests

THE PROMPT

You are a senior security-focused full-stack engineer specializing in production authentication systems.

CONTEXT:

Follow PRD.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md.

TASK:

Implement secure authentication and authorization for this project.

Cover:

- Signup/login/logout where required
- Session/token management
- Password handling if applicable
- Protected routes and APIs
- User roles and permissions
- Resource ownership checks
- Unauthorized/forbidden states
- Session expiry and secure handling
- Relevant tests

RULES:

Use the authentication approach defined in ARCHITECTURE.md. Never store passwords or tokens insecurely. Never trust client-side authorization. Enforce permissions server-side. Do not expose secrets. Do not replace existing auth without justification.

OUTPUT:

Implement, test and report the authentication flow, protected resources, permissions, changed files and any security concerns.

MCP CONNECTION

Use this when your coding environment needs an MCP server, for example to reach a database or a design tool. It asks the agent to enable only the tools the project needs.

READS	YOU FILL IN	YOU GET
AGENTS.md and the existing architecture	[MCP SERVER / PURPOSE]	A verified MCP connection

THE PROMPT

You are a senior AI engineer experienced with MCP servers and AI coding agents.

CONTEXT:

The project requires this MCP capability:

[MCP SERVER / PURPOSE]

Follow AGENTS.md and the existing project architecture.

TASK:

Connect and configure the required MCP server for the coding environment.

First determine:

- What tools/resources the MCP exposes
- Which capabilities this project actually needs
- Required configuration and permissions
- Required environment variables or credentials

Then configure the connection and verify that the required tools work.

RULES:

Follow least-privilege access. Never expose credentials or commit secrets. Do not enable unnecessary tools. Do not modify application architecture unless required. Explain any permissions or configuration that require manual approval.

OUTPUT:

Report the MCP connection status, available relevant tools, configuration performed, verification results and any manual steps required.

EXTERNAL API / SERVICE INTEGRATION

Use this for each outside service the product depends on, such as payments, email or an AI provider. Run it once per service.

READS	YOU FILL IN	YOU GET
API.md, ARCHITECTURE.md, DATABASE.md, AGENTS.md	[SERVICE / API NAME + PURPOSE]	A tested integration

THE PROMPT

You are a senior integration engineer experienced in building reliable production integrations.

CONTEXT:

Follow API.md, ARCHITECTURE.md, DATABASE.md and AGENTS.md.

TASK:

Integrate this external service:

[SERVICE / API NAME + PURPOSE]

Implement:

- Secure authentication/configuration
- Required API operations
- Request/response handling
- Input/output validation
- Error handling and timeouts
- Rate-limit handling where relevant
- Retries only where safe
- Webhooks and signature verification if required
- Logging without exposing sensitive data
- Tests for success and failure cases

RULES:

Never expose API keys client-side or hardcode secrets. Follow the provider's documented API behavior. Make failures graceful and prevent duplicate operations where relevant. Do not add unrelated functionality.

OUTPUT:

Implement and verify the integration. Report configuration, environment variables required, files changed, tests performed and any remaining limitations.

CHAPTER 4 · PROMPTS 14–17

TEST, DEBUG AND HARDEN

Four prompts that check the work. Testing and security audits ask the agent to investigate failures rather than hide them. Debugging demands a root cause before any fix is made. Code review changes only what is clearly justified and safe.

Run these once a meaningful amount of the product exists, and again before each release.

IN THIS CHAPTER

14 Testing & QA	24
15 Debugging / Root Cause	25
16 Security Audit & Hardening	26
17 Code Review & Refactoring	27

PROMPT 14 · CHAPTER 4

TESTING & QA

Run this once a set of features exists. It tests the journeys that matter most first, then edge cases, permissions and failure states.

READS	YOU FILL IN	YOU GET
PRD.md, API.md, ARCHITECTURE.md and the current code	Nothing	Tests, and a report of passes, failures and bugs found

THE PROMPT

You are a senior QA engineer and test automation specialist responsible for production software quality.

CONTEXT:

Read PRD.md, API.md, ARCHITECTURE.md and the current implementation before testing.

TASK:

Create and execute a practical test strategy for the implemented functionality.

Test:

- Core user journeys
- Happy paths
- Edge cases
- Invalid inputs
- Authentication and permissions
- API/database failures
- Loading, empty and error states
- Important integrations
- Responsive behavior where relevant

Add appropriate unit, integration and end-to-end tests for critical behavior.

RULES:

Prioritize business-critical paths over superficial coverage. Do not modify production behavior just to make tests pass. Investigate failures rather than masking them.

OUTPUT:

Report tests created/run, passed and failed cases, bugs discovered, severity, and recommended fixes.

DEBUGGING / ROOT CAUSE

Paste the error, the log or a description of the screenshot. The prompt forces a trace to the real cause before any fix is made.

READS	YOU FILL IN	YOU GET
AGENTS.md and the existing code	[ERROR / BUG / LOG / SCREENSHOT]	Root cause, smallest fix and a regression check

THE PROMPT

You are a senior debugging engineer responsible for diagnosing production-quality software issues.

CONTEXT:

Follow the project's existing architecture, code and AGENTS.md rules.

PROBLEM:

[ERROR / BUG / LOG / SCREENSHOT]

TASK:

Investigate the issue systematically.

1. Reproduce or trace the failure.
2. Inspect relevant code, logs, data and dependencies.
3. Identify the actual root cause.
4. Explain why it happens.
5. Implement the smallest correct fix.
6. Test the fix and check for regressions.

RULES:

Do not guess or randomly patch code. Do not hide errors, weaken validation, remove tests, or change unrelated functionality. Preserve existing behavior unless the bug requires otherwise. If evidence is insufficient, state what is missing.

OUTPUT:

Report root cause, fix implemented, files changed, tests performed and any remaining risk.

SECURITY AUDIT & HARDENING

This audits authentication, secrets, input handling, uploads and data exposure, highest-impact issues first, and fixes what it finds.

READS	YOU FILL IN	YOU GET
PRD.md, ARCHITECTURE.md, DATABASE.md, API.md, AGENTS.md	Nothing	Findings by severity, and fixes

THE PROMPT

You are a senior application security engineer specializing in production web applications.

CONTEXT:

Audit the current project using PRD.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md.

TASK:

Perform a security audit and fix vulnerabilities you find.

Check:

- Authentication and authorization
- Secrets and environment variables
- API and database access
- Input validation and sanitization
- Injection and XSS risks
- CSRF where applicable
- File uploads
- Rate limiting and abuse
- Sensitive data exposure
- Dependencies
- Error handling and logging
- Client/server trust boundaries

RULES:

Do not weaken functionality or security controls to hide issues. Do not expose secrets. Prioritize exploitable and high-impact issues first. Make fixes minimal and compatible with the existing architecture.

OUTPUT:

Report findings by severity, fixes implemented, tests performed, and any risks that remain.

CODE REVIEW & REFACTORING

A pre-release review against your own documents. It fixes only what is clearly justified and safe, so working code is not rewritten for style.

READS	YOU FILL IN	YOU GET
All six documents	Nothing	Review findings and safe refactors

THE PROMPT

You are a senior software engineer reviewing a production codebase before release.

CONTEXT:

Review the current implementation against PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md.

TASK:

Perform a thorough code review for:

- Architecture violations
- Bugs and incorrect logic
- Duplicate or dead code
- Poor abstractions
- Overly complex code
- Inconsistent patterns
- Unnecessary dependencies
- Poor error handling
- Maintainability issues
- Technical debt
- Performance concerns

Fix issues that are clearly justified and safe to change.

RULES:

Do not rewrite working code merely for style. Preserve behavior and follow existing architecture. Do not introduce unnecessary abstractions or dependencies. Test important changes.

OUTPUT:

Report critical findings, refactoring performed, files changed, tests run and remaining technical debt.

CHAPTER 5 · PROMPTS 18–20

SHIP AND WATCH

Three prompts take the product live and keep it healthy: deployment, monitoring and a final audit. The audit ends with a verdict, and its rules say not to assume something works just because the code exists.

Treat the verdict as input to your own decision, and verify by hand the items it lists before launch.

IN THIS CHAPTER

18 Deployment & Production Setup	29
19 Production Monitoring & Maintenance	30
20 Final Production Audit	31

DEPLOYMENT & PRODUCTION SETUP

This prepares the production build, environment, database migrations, HTTPS and health checks. Anything that needs your approval is flagged.

READS	YOU FILL IN	YOU GET
ARCHITECTURE.md, API.md, DATABASE.md, AGENTS.md	Nothing	A verified deployment, or a deployment-ready setup

THE PROMPT

You are a senior DevOps engineer experienced in deploying secure production applications.

CONTEXT:

Follow ARCHITECTURE.md, API.md, DATABASE.md and AGENTS.md.

TASK:

Prepare this application for production deployment.

Configure and verify:

- Production build
- Hosting/deployment environment
- Environment variables and secrets
- Production database
- Database migrations
- Domain and HTTPS where applicable
- CORS and production URLs
- Build/start commands
- Error handling
- Required external services
- Health checks
- Production configuration

RULES:

Never commit secrets or use development credentials in production. Do not destroy existing data. Verify the production build before deployment. Follow the architecture's deployment strategy and flag anything requiring manual approval.

OUTPUT:

Deploy or prepare the deployment, verify it, and report configuration, environment variables required, migration status, deployment URL/status and any remaining manual steps.

PROMPT 19 · CHAPTER 5

PRODUCTION MONITORING & MAINTENANCE

This adds error tracking, logs, uptime checks and alerts sized to your project, without collecting unnecessary user data.

READS	YOU FILL IN	YOU GET
ARCHITECTURE.md and the deployed application	Nothing	Monitoring, alerts and a list of remaining gaps

THE PROMPT

You are a senior SRE responsible for keeping production applications reliable and observable.

CONTEXT:

Review the deployed application and its ARCHITECTURE.md before making changes.

TASK:

Set up practical production monitoring for this application.

Cover where applicable:

- Application errors and exceptions
- Server/API logs
- Database health
- Uptime and availability
- Performance
- Failed external integrations
- Background jobs
- Resource usage
- Critical user/business events
- Alerts for actionable failures
- Backups and recovery verification

RULES:

Do not collect unnecessary sensitive user data. Avoid noisy alerts. Never log secrets, passwords or sensitive tokens. Prefer simple monitoring appropriate for the project's actual scale.

OUTPUT:

Implement or configure the required monitoring and report what is tracked, alert conditions, backup/recovery status, and remaining observability gaps.

PROMPT 20 · CHAPTER 5

FINAL PRODUCTION AUDIT

The last gate. It checks the whole project against the documents and returns one of three verdicts.

READS	YOU FILL IN	YOU GET
All six documents	Nothing	READY, READY WITH WARNINGS or NOT READY

THE PROMPT

You are a senior engineering lead responsible for approving an application for production launch.

CONTEXT:

Audit the complete project against PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md.

TASK:

Perform a final end-to-end production readiness audit.

Verify:

- All critical requirements and user journeys
- UI/UX consistency
- Functional correctness
- Authentication and authorization
- Database integrity
- API and integrations
- Security
- Tests and regressions
- Performance
- Responsive behavior
- Accessibility
- Error/loading/empty states
- Environment and deployment configuration
- Monitoring, backups and recovery

RULES:

Do not assume something works because the code exists. Verify it where possible. Do not make risky changes automatically. Prioritize blockers and high-impact issues.

OUTPUT:

Give a launch verdict: READY, READY WITH WARNINGS, or NOT READY. List findings by severity, required fixes, tests performed, and anything that must be manually verified before launch.

QUICK REFERENCE

All twenty prompts on one page.

#	PROMPT	YOU FILL IN	YOU GET	PAGE
01	PRD Creation	[PROJECT IDEA]	PRD.md	08
02	DESIGN.md	—	DESIGN.md	09
03	ARCHITECTURE.md	—	ARCHITECTURE.md	10
04	DATABASE.md	—	DATABASE.md	11
05	API.md	—	API.md	12
06	AGENTS.md / Coding Rules	—	AGENTS.md	13
07	Implementation Planning	—	An ordered implementation plan	15
08	Project Setup	—	A runnable project skeleton	16
09	Feature Development	[TASK / FEATURE]	A tested feature and a change report	18
10	Database & Backend Connection	—	A working backend and database	19
11	Authentication & Authorization	—	Authentication, authorization and tests	20
12	MCP Connection	[MCP SERVER / PURPOSE]	A verified MCP connection	21
13	External API / Service Integration	[SERVICE / API NAME + PURPOSE]	A tested integration	22
14	Testing & QA	—	Tests, and a report of passes, failures and bugs found	24
15	Debugging / Root Cause	[ERROR / BUG / LOG / SCREENSHOT]	Root cause, smallest fix and a regression check	25
16	Security Audit & Hardening	—	Findings by severity, and fixes	26
17	Code Review & Refactoring	—	Review findings and safe refactors	27
18	Deployment & Production Setup	—	A verified deployment, or a deployment-ready setup	29
19	Production Monitoring & Maintenance	—	Monitoring, alerts and a list of remaining gaps	30
20	Final Production Audit	—	READY, READY WITH WARNINGS or NOT READY	31

CONCLUSION

Twenty prompts will not build a product by themselves. What they do is take away the guessing: the agent always has a document to consult, a rule to follow and a report format to return.

Start with one small idea. Run the first six prompts, read what comes back, and fix what is wrong while it is still a paragraph and not a codebase. Then let the documents carry the work from there, one task at a time.

DOCUMENTS **WRITE THEM FIRST.** Scope, design, architecture, data, API and rules.	**RULES** **KEEP THE AGENT INSIDE THEM.** No invented features, no secrets in code, no unrelated changes.	**EVIDENCE** **ASK FOR REPORTS.** Every prompt ends by asking what changed, what was tested and what is left.

Then start small. One idea, one feature at a time, with the documents open, will teach you more in two weeks than any book can.

ABOUT THIS BOOK

TWENTY PROMPTS. ONE WORKFLOW.

This book collects twenty prompts that take a software project from a one-line idea to a production launch with an AI coding agent: documents first, then planning, building, testing and shipping.

Five chapters follow the order the work happens: lay the foundations, plan and set up, build the product, test, debug and harden, ship and watch.

Replace the square-bracket placeholders, paste each prompt as written, and read every output before you approve it.